IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 24 October 2024, accepted 19 November 2024, date of publication 25 November 2024, date of current version 4 December 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3505298

RESEARCH ARTICLE

Leveraging Local LLMs for Secure In-System Task Automation With Prompt-Based Agent Classification

SUTHIR SRIRAM[1], C. H. KARTHIKEYA[1], K. P. KISHORE KUMAR[1], NIVETHITHA VIJAYARAJ[1], AND THANGAVEL MURUGAN[2], (Senior Member, IEEE)

[1]Department of Computer Science and Engineering, Amrita School of Computing, Amrita Vishwa Vidyapeetham, Chennai 601103, India
[2]College of Information and Technology, United Arab Emirates University, Al Ain, United Arab Emirates

Corresponding author: C. H. Karthikeya (karthikeya2204@gmail.com)

This work was supported by Amrita Vishwa Vidyapeetham.

ABSTRACT Recent progress in the field of intelligence has led to the creation of powerful large language models (LLMs). While these models show promise in improving personal computing experiences concerns surrounding data privacy and security have hindered their integration with sensitive personal information. In this study, a new framework is proposed to merge LLMs with personal file systems, enabling intelligent data interaction while maintaining strict privacy safeguards. The methodology organizes tasks based on LLM agents, which apply designated tags to the tasks before sending them to specific LLM modules. Every module is has its own function, including file search, document summarization, code interpretation, and general tasks, to make certain that all processing happens locally on the user's device. Findings indicate high accuracy across agents: classification agent managed to get an accuracy rating of 86%, document summarization reached a BERT score of 0.9243. The key point of this framework is that it splits the LLM system into modules, which enables future development by integrating new task-specific modules as required. Findings suggest that integrating local LLMs can significantly improve interactions with file systems without compromising data privacy.

INDEX TERMS File system, few-shot prompting, LangChain, LLM, prompt engineering.

I. INTRODUCTION

In today's world of growing data and increasingly advanced personal computing setups, managing and navigating personal file systems has become more intricate. At the same time, Large Language Models (LLMs) are making impressive progress in understanding and generating natural language, transforming interactions with digital systems to boost productivity and simplify data handling. Personal file systems are crucial for digital experiences as they contain a wide range of sensitive information from coding projects to financial records. The potential for LLMs to interact intelligently with these file systems opens up opportunities for productivity, better data organization and more intuitive data management that adapts to context.

The associate editor coordinating the review of this manuscript and approving it for publication was Arianna Dulizia.

Recent research has explored methods to enhance interactions with documents using artificial intelligence. Studies have investigated the use of Large Language Models to improve document summarization, highlighting their efficiency and accuracy [1]. Moreover, they explored how LLMs can aid in code generation [2] and summarization [3], highlighting their capacity to streamline the development processes and minimize errors. Newer models such as like Claude Sonnet 3.5 [4] and GPT-4o [5] excel at these tasks, but none work with the users personal file system. Moreover, these methods frequently depend on processing or necessitate transmitting data to external servers, which brings up privacy issues [6] and restricts their feasible use in personal computing settings.

This research intends to accomplish three principal objectives in reaction to the identified challenges. The first step is to develop a privacy-focused framework, permitting

© 2024 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

local processing of personal information by large language models (LLMs), so that sensitive data remains safely on the user's device and is not sent externally. Second, to develop a modular architecture that divides tasks into individual modules for separate operations–like file search, document summarization, code debugging and delivering flexibility for foreseeable growth. At last, analyze the operation of the proposed framework in terms of accuracy, speed, and user satisfaction, all while protecting privacy strongly and preserving computational efficiency. The key components of the framework includes:

- *Prompt-Based Agent Classification:* A system that uses LLM agents to classify user tasks to appropriate modules.
- *Natural Language File System Interface:* A component that converts natural language queries into Shell commands and retrieve the files requested.
- *Context-Aware Retrieval System:* A retriever that fetches relevant past conversations to enhance the context for current conversation if needed.
- *Specialized Task Modules:* LLM-powered modules for various tasks including file search, document processing, code debugging, and general task.

In testing the system, it's performance was evaluated in terms of accuracy, speed and user satisfaction. The findings show enhancements in the way the system interacts with files while still ensuring strong privacy safeguards.

The rest of this paper is structured as follows. Section II delves into an examination of existing research. Section III outlines the proposed framework and its main components. Experimental setup and evaluation metrics are presented in Section IV. Section V discusses the key findings of the evaluation and Section VI wraps up the paper. Lastly, Section VII Suggests paths for future research.

II. RELATED WORKS

The large language models (LLMs) have made great strides in the fields of code generation, document summarization, question-answering systems, and retrieval-augmented generation of information in recent years. In one approach, Li et al. [7] investigated the use of ChatGPT for rapid source code development that involves a web-based code generation platform made up of User Interface, Prompt Builder and Backend Service. There were several metrics indicating substantial improvements due to this technique which enhances model performance through dynamic prompt generation including a 65.06% improvement in exact match (EM) and a 50.64% improvement in Pass@1. This has shown how well ChatGPT can be used in real development scenarios as confirmed by Wang et al. [8], who evaluated prompt tuning in code intelligence tasks. For instance, prompt tuning outperforms traditional fine-tuning consistently especially on low resource cases where prompt tuning obtained an extra 26% BLEU scores for code summarization tasks showing how effective it is in improving the performance of pre-trained models on coding-related tasks.

Zhong and Wang et al. [9] question the robustness and reliability of LLMs in generating code, examining whether they can replace traditional sources like Stack Overflow. In this study, the ROBUSTAPI dataset is introduced to assess the reliability of the generated code and expose common misuse patterns. It emerged that even advanced models such as GPT-4 have a high percentage of API misuses in the code produced by them. More resilient evaluation frameworks are needed for LLM-generated code. Reference [10] did a study on the way LLMs handle distracting inputs. The results suggest that contemporary language models are not able to accurately distinguish between very similar concepts and often confused by such interferences. Moreover, other mitigation techniques like those for dealing with the question of extraneous information fail to enhance LLM immunity from this kind of distractors hence calling for better ways of minimizing their effect on LLM reactions.

For instance, Lu et al. [11] found that one of the most challenging tasks for long documents summarization by LLMs is their ability to summarize long texts. Proposed hybrid ChatGPT-based method combined with an extractive C2F-FAR summarization model yield competitive results with automated scores but there are several concerns about text coherence and faithfulness which implies that despite promising nature of ChatGPT type LLMs, they cannot be absolutely relied upon when it comes to summarizing lengthy texts. The case also covers book-length document summary discussed by Chang et al. [12], who proposed a novel metric BOOKSCORE to measure the coherence of summaries generated by LLMs. In [13], KEIT-Sum, a key-element-informed instruction tuning method is described which enhances relevancy in sLLMs' summaries generation process. Besides reducing hallucinations, KEIT-Sum increases summary quality to compete with proprietary LLMs on relevance only.

Hicke et al. [14] address the development of intelligent question-answer systems by introducing AI-TA, a QA assistant that leverages open-source LLMs from the LLaMA-2 family. The study combines retrieval-augmented generation (RAG) with supervised fine-tuning and direct preference optimization to achieve 30% increase in answer quality and showcase how open-source LLMs can be used in education, as well as cultivate scalable and smart QA systems. Furthermore, Jiang et al. [15] delve into enriching LLMs with retrieval-augmented generation and propose the FLARE model for active retrieval during text generation. FLARE outperforms baseline models in long-form knowledge-intensive tasks by iteratively predicting and retrieving relevant information, thus indicating that continuous information gathering throughout the process is effective. Moreover, SKR (Self-Knowledge guided Retrieval) [16] effectively utilizes LLMs' self-knowledge to determine when external resources are needed, improving the accuracy of responses. In the same realm SlimPLM [17] introduces a

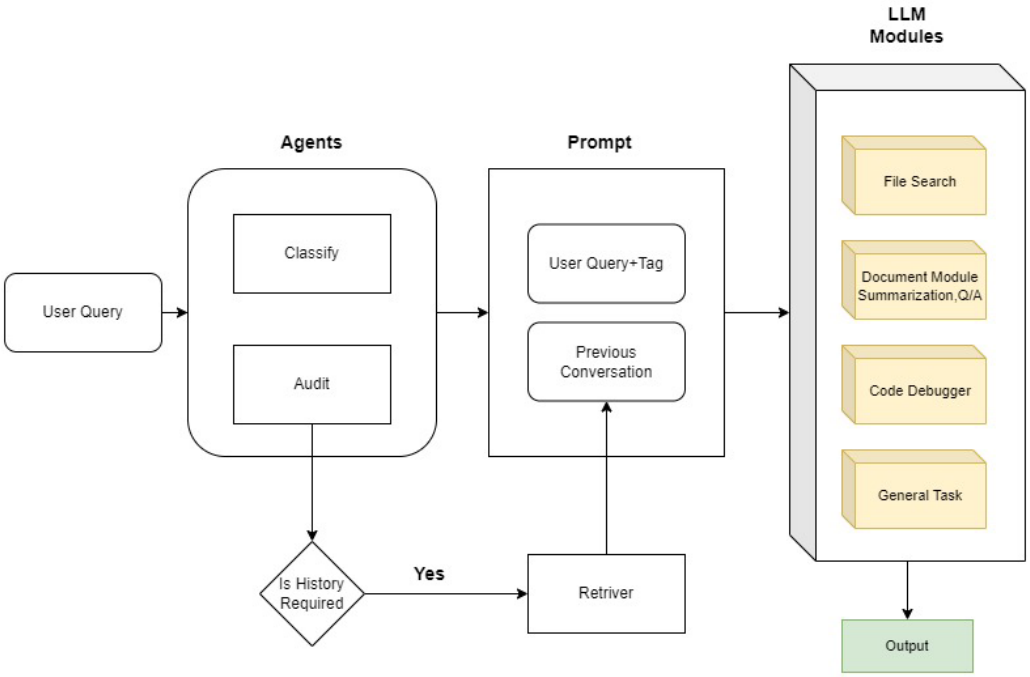


FIGURE 1. System architecture for handling user queries.

new way to deal with knowledge retrieval and optimization in LLMs using a slim proxy model for efficient detection of missing knowledge. This slim proxy model, which has fewer parameters compared to standard LLMs, gives rules-of-thumb that help identify what the LLM knows versus what it does not. In SlimPLM however, only very few computational costs are incurred by answering queries related to missing information, saving up on computations and increasing end-to-end performance on question answering tasks in LLMs.

MobileGPT1 is a model developed by Lee et al. [18], which combines LLMs with human-like memory for mobile task automation. It mimics human cognitive processes by breaking tasks down into smaller sub-tasks for better learning and implementation precision. Also, Brown and Taylor [19] present MemGPT, which is responsible for managing virtual context in order to overcome the limitations of inherent context windows solely used in LLMs. This strategy is convenient when dealing with long documents as well as multi-session conversations, which have made it possible to optimize the performance of LLMs in these areas. In addition, there is LONGMEM presented by Wang et al. [20], a framework whose objective is to incorporate long-term memory capabilities into LLMs. In contrast to the traditional memories that cannot cache or update long-term contexts, such functionalities are demonstrated by LONGMEM through its employment of separate network architectures while doing so leading to significant advances in memory-augmented tasks.

Reference [21] introduces Sahaay, an innovative framework, in relation to automating customer service. It is a versatile approach that can apply across industries and offer instant help as well as query resolution through tactics like using web scraping or fine-tuning. By making customer interactions efficient, individualized and prompt, Sahaay has succeeded in redefining the relationship between customers and companies by increasing its own retention rate and brand equity.In the same realm [22] present an upgraded system for developing large-scale model applications based on LangChain+ChatGLM+Rasa which connects fine-tuned AI models with local vector data sources. This facilitates building intelligent applications that have modular abstractions and customization capabilities; thus reducing development time and costs. Reference [23] highlights accelerated growth in applications requiring LLMs to be built from LangChains. It demonstrates how the core features of LangChain can facilitate modular abstractions and customizable pipelines for fast development of LLM-based applications.

Vector database is another field looked into by the author Han et al. [24]. The article also provides different ways through which vector databases can be merged with LLM to yield novel possibilities. In particular, this paper looks into algorithms pertinent to storing high-dimensional data as well as related challenges as a way of categorizing means employed to tackle approximate nearest neighbor search problem.

In the context of multi-agent systems, Händler [25] investigate autonomy versus alignment in LLM based architectures. This involves breaking down hard problems into smaller sub-tasks handled by specialized agents. The taxonomy suggested serves as an analytical tool that covers diverse

TABLE 1. Performance comparison of models across various benchmarks.

Model	Modality	MMLU	HellaSwag	Winogrande	Arc-c	HumanEval	MBPP	Math	GSM8K
CodeLlama 7B	Pretrained	36.9%	62.9%	62.3%	34.5%	31.1%	52.5%	5.2%	20.8%
Qwen2 7B	Pretrained	70.3%	80.7%	77.0%	60.6%	51.2%	65.9%	44.2%	79.9%
Mistral 7B	Pretrained	64.2%	83.2%	78.4%	60.0%	30.5%	51.1%	13.1%	52.2%

factors such as autonomy and alignment dynamics in these systems hence contributes to better designed multilateral agent's architecture. Lastly, [26] focuses on multi-agent systems that can be used to make LLMs better through collaboration between multiple intelligent agents that take part in complex tasks and highlights its practicality and flexibility with case studies in AGI.

While today's methods investigate the possibilities of local LLM processing, they do not furnish a modular and extensible framework capable of integrating with personal file systems. Present solutions center around particular, separated tasks—such as summarization or code debugging—but are inadequate in supplying a system that can address a broader range of tasks, including natural language-based file search, contextual retrieval, and general document related tasks to name a few. In addition, the designed solutions are difficult to scale, thus creating a large gap in forming a comprehensive, privacy-preserving framework that works smoothly with personal file systems.

This paper focuses on the use of local LLMs in implementing some of these concepts for in-system LLM tasks on the file system. The aim is to improve data privacy and integrity by utilizing local LLMs when investigating the operationalizations of sophisticated language models in a secure and efficient manner.

III. METHODOLOGY

This research examines how the integration of large language models (LLMs) with personal file systems could improve data privacy and integrity. The methodology section is composed of several parts including Model selection, Agents, Retriever and LLM modules.

The proposed framework is modular design coupled with its ability to perform processing tasks on the device. Differing from existing systems which concentrate on individual tasks, this approach combines various specialized modules, powered by LLMs, to provide a multitude of functions—like natural language file search, document summarization, and code debugging—with one framework.

Fig 1 depicts the overview of the developed system. Firstly both agents receive the user query initially. User query gets tagged into a category by classification agent. The user query is audited by an auditing agent to determine whether the contexts happen to be outside of the context window, if so then the query would be directed towards a retriever module that fetches older conversations and it will be posted along with user query otherwise the input is sent to its respective LLM modules based on its classification. After execution, reply is sent back to the user.

A. MODEL SELECTION

The research required a Large Language Model (LLM) meeting some specific criteria. Three models were examined – qwen2 [27], Mistral 7B [28], codeLlama 7B [29]. Criteria for selecting the model were as follows:

- *Parameter Size:* The parameter count of the model should be around 7 billion so that it can work with different computational resources.
- *Prompt Sensitivity:* It had to be possible to depend on the model in order to get accurate and reliable outputs by always following system prompts.
- *Controlling Extended Contexts:* The objective of the model was to handle even longer contexts allowing complex activities like summary of documents in long-form.
- *Uniqueness in Task Performance:* To cater for different needs the LLM had to have abilities both in coding and non-coding tasks.

After a comprehensive comparison (Table 1) with other models, Mistral 7B turned out that it is the most suitable model. It has 7 billion parameters and delivers a balance between the speed of calculations and the performance of the work, which means it is suitable for devices with different computational capabilities. In testing, Mistral 7B sticked to system prompts giving reliable outcomes that are consistent with described guidelines. The model's context length of 8,000 tokens proves its suitability for the processing of long inputs, including document summarization and code generation, where input queries are long. CodeLlama 7B although competent enough at coding tasks, but it did not perform as effectively on non-coding tasks or generalized scenarios. Furthermore, Qwen which scored higher than Mistral 7B in benchmarks was seen to produce more hallucinations than Mistral 7B when generally tested, hence cannot be relied upon. In terms of memory requirements and processing power, all the compared models have 7 billion parameters, meaning their hardware demands are roughly similar. In conclusion, Mistral seemed to be the right choice for the research.

B. AGENTS

LLM agents are autonomous components that use the capabilities of advanced language models to do a task. They are designed to process and interpret natural language inputs and use pre-defined rules and examples to perform classification, summarize, retrieve data and so on. Using few shot prompting or fine tunded models, LLM agents can adapt

to user queries of any kind and produce accurate, contextually meaningful outputs.

LLM agents are used at the core of the framework. User inputs go through them, they understand what the user is asking for and activate the right internal modules (summarization or file retrieval, for example). These agents are independent, with prompt strategies and knowledge in the model that allows for high levels of accuracy and task specific performance.

1) CLASSIFICATION AGENT

The Classification Agent operates by evaluating the natural language input provided by the user. In order to establish what exactly a request means, it employs a prompt-based analysis approach [30], [31]. As an example, a ''code debugging'' task is identified when a user makes an inquiry relevant to debugging code in the system. Consequently, this agent outputs a structured response that consists of a dictionary with category tags as keys and original prompts from users as values.

Algorithm 1 Task Classification

```
procedure classify(query)
    prompt ←
     Please classify the following query into one of the
        categories:
    - FS for File Search
    - SA for Document Summary and Q/A
    - CD for Code Debugging
    - GT for General Task
    Query: query
    Output format: Category Tag
    return prompt
    function invoke(prompt,agent)
        output ← agent.invoke(prompt)
        return output.strip()
        prompt ← format_prompt(user_query)
        category_tag ← inv_agent(prompt, agent)
        return category_tag
    end function
end procedure
```

A well-structured prompt is used by the algorithm to help LLMs categorize user queries. Afterward, the output from the agent is employed in redirecting the query towards a suitable LLM module. Let us say, for example that queries labeled with ''FS'' would be dealt by file searches responsible module or those marked ''SA'' would be functioned by summarization and Q/A specialized module. Classification is done using Mistral 7B.

There are a few cases where a query is misclassified by the agent. This setup does not a traditional feedback loop where the model learns over time dynamically, but user interactions are a kind of real-time error correction. The input query can be rerun by the user with the correct tag, so the task gets handled by the correct module. An example would be a query like ''Summarize file.txt /SA'' would be sent to the document summarization module.Few-shot prompting is used, feeding the model with several examples to guide how to classify or process tasks.

The system utilizes few-shot prompting, multiple examples are provided to guide the model in classifying and processing tasks. The model is able to generalize better over a wide range of user inputs, and as a result, this approach greatly reduces the frequency of misclassified queries. This leads to less errors happening, and overall the system requires less user intervention, directly reducing the cognitive load on users.User intervention is preserved for more complex or ambiguous cases, and the few shot prompting strategy has significantly reduced misclassifications. Future enhancements may include true feedback loops like adding the corrected tag and respctive prompt to the example such that the system can autonomously adjust prompt strategies or parameter fine tuning via user corrections.

There are few cases where a query could be categorized into multiple tags, to avoid that the classification agent employs a specific approach to understand the user's input by evaluating its content and categorizing it according to the prompt to counter any undesired outcomes. And the prompt instruction has been given to only generate one tag at max. This helps direct the task to the most suitable module. If there is uncertainty (for example here, a query can be classified as belonging to one or more categories), the system works towards the better option from what has been given by the query.

2) AUDIT AGENT

This agent is crucial in controlling the conversation context of the system, particularly when it goes beyond the Context Window of LLM. The agent ensures that interactions remain orderly and controllable by effectively dealing with overflow. An instance where an Auditing Agent is triggered whenever the length of conversation stored in cache file passes over some predefined token limit.

To begin the Read cache function, the specified file is opened and the entire conversation in it is read to retrieve its contents. Secondly, the token count function calculates how many tokens are there in that conversation cache. This helps establish whether or not this conversation size exceeds the LLMs content length (32K). In case of a conversation overstepping CONTEXT_WINDOW_LIMIT, the Agent calls initiate_retrieval which sends current user query to Retriever module for further
processing.

C. RETRIEVER

Retriever fetches appropriate previous conversations to add context and coherence in user interactions. By doing this, it ensures that Large Language Model (LLM) modules attain the most important details that are required for processing the current query. The retriever deals with huge historical

Algorithm 2 Auditing logic

```
Input: path, query
Output: None (Side effect: Initiates retrieval if necessary)
Constant: CONTEXT_WINDOW_LIMIT ← 32000
function read_cache(file_path)
    file ← openfileatfile_path
    return content of file
    prompt ← format_prompt(user_query)
    category_tag ← invoke_agent(prompt, agent)
    return category_tag
end function
function manage_context(path,query)
    conv_cache ← read_conv_cache(path)
    conv_cache ← wordsinconv_cache
    if token_count > CONTEXT_WINDOW_LIMIT then
        initiate_retrieval(user_query)
    end if
end function
```

conversation data by using Sentence transformers model for embedding generation and FAISS library for efficient similarity searches.

Fig 2 depicts the flow of the retriever model.

To begin, it starts by loading cached conversation data from a file made up of previous user interactions stored on a line-by-line basis and handled by load_text_file function, which ensures that only non-empty lines are taken into account. Then the Sentence Transformers library's [32] 'all-MiniLM-L6-v2' model is utilized to generate embeddings for the conversation data. This model was chosen for its combination of efficiency and accuracy. To this effect, using generate_embeddings function produces embeddings for each line of the cached conversation data which captures the semantic meaning in text. In order to enable efficient similarity searches, the retriever uses FAISS (Facebook AI Similarity Search) library to build an index of embeddings. The build_faiss_index function initializes and populates this index. On receiving a new user query, search function will encode it as an embedding and perform similarity search against FAISS index. Based on distances in embedding space [33], the top-k [34] most similar past conversations are retrieved giving LLM enough historical context to produce accurate and coherent answers.

Semantic search ensures the relevance of interactions in historical queries scenarios. This approach compares the input query to the history of previous conversations in order to find matching conversations that are meaningful, not just keyword based. Self Knowledge Retrieval (SKR) is used as a safeguard, in rare cases where irrelevant data might still be retrieved. During the final parsing of output, SKR checks whether the retrieved information is sufficient to answer the query given. If the retrieved data is contextually not related to the input query, it instead generates a response using its own knowledge base.

Algorithm 3 Semantic Search

```
Input: query, text_data, model, k
Output: results
function load_text_file(file_path)
    text_data ← empty list
    for each line in file at file_path: do
        if line is not empty: then
            add stripped line to text_data
        end if
    end for
    return text_data
end function
function generate_embedding(text_data)
    return model.encode(text_data)
end function
function build_faiss_index(embeddings)
    dimension ← number of columns in embeddings
    index ← create FAISS index with dimension
    add embeddings to index
    return index
end function
function search(query,index,text_data,model,k):
    query_embedding ← model.encode(query)
    distances, indices ←
        index.search(query_embedding, k)
    results ← empty list
    for each i in indices: do
        add text_data[i] to results
    end for
    return results
end function
```

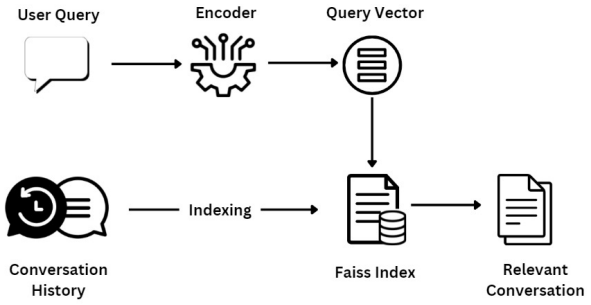


FIGURE 2. Architecture for retriever module.

D. LLM MODULES

The successful implementation of system is largely dependent on specialized LLM modules, which are all aimed at performing specific types of tasks. These modules are the backbone of a smooth-running system and allow for an accurate and timely response to user queries.

1) FILE SEARCHING MODULE

The File Searching Module is created to respond to searches where users wish to locate or retrieve files from their systems.

Once a query has been entered by the user, it is translated into a PowerShell/cmd command by the LLM. This translation employs LLM's natural language understanding abilities in order to ensure that the query is correctly converted into an executable command. The terminal then begins an instance before running this command and retrieving the requested file. It's common for users to search through a series of file systems and remember the necessary file paths, which is often time-consuming and unproductive, particularly in the case of large directories or complex file structures. The module eliminates such manual work by transforming natural language queries into PowerShell and cmd commands, hence allowing for faster and less tedious file retrieval. Besides, the use of LLMs makes it possible to input vague and incomplete requirements. For example, even users who cannot remember the precise name or location of a file are able to retrieve it based on a description of the contents or approximate name of the file.

2) DOCUMENT MODULE

This module handles a range of activities touching on document processing; it provides summaries, answers questions and even long-form summarization. It does this with the help of Mistral 7B's strong natural language processing. For Summarization, the module produces brief descriptions for documents that capture the most important aspects while leaving out irrelevant information.Q/A is when people ask questions and they get answers based on the contents of their desired documents in a system.Fig 3 shows how Long-Form Summarization is handled [35]. Given very long texts (the limit is 8,000 tokens), this system divides them into parts that are equal in length according to context. Thereafter, it offers overviews of all these sections and then goes ahead to give an integrated overall summary for such pieces.

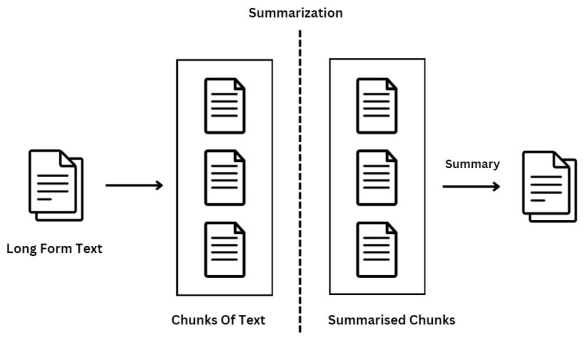


FIGURE 3. Long form text summarization.

3) CODE DEBUGGER

This module leverages the native capabilities of Mistral 7B to assist in debugging code. It automatically detects errors in programs written in various programming languages and offers recommendations for their correction. Additionally, it analyzes code to provide insights. The use of appropriate LLMs for this task is crucial, as they have been trained on a wide range of codebases and debugging scenarios, which are essential for developers who need fast and accurate debugging assistance.

4) GENERAL TASKS

There is the General Tasks Module that handles all kind of assignments besides the particular types covered by other modules. This module also makes use of some of Mistral 7B's intrinsic capabilities in order to understand and respond to a wide variety of questions. To make it reliable, Self-Knowledge Retrieval (SKR) has been implemented, [16], [36]mechanism. By this, the LLM internally checks if it has adequate knowledge on any query asked before answering. In case the LLM concludes that it does not possess sufficient information for giving an accurate response, instead of producing potentially incorrect or misleading information, it will say ''I don't know''. Here hallucinations are reduced to a minimum and thus making sure the answers are reliable.

IV. EVALUATIONS

The aim of evaluation is to address the research questions below:

RQ1: Does the agent classification system efficiently and accurately categorize tasks for execution?

RQ2: How effective is the retriever module in identifying and retrieving pertinent past interactions to aid present task responses?

RQ3: Compared to expected benchmarks, how does overall system performance in terms of document summarization and how does it fare in terms of consistency, relevance and user satisfaction?

A. EVALUATION METHODS

As for evaluating the performance of a given system, several metrics and methodologies were used:

1) ACCURACY

Accuracy is a metric that determines the percentage of tasks classified correctly into pre-defined categories. High accuracy ensures correct categorization of tasks necessary for reliable performance.

$$\text{Accuracy} = \frac{\text{Number of Correctly Classified Tasks}}{\text{Total Number of Tasks}} \quad (1)$$

2) AVERAGE RESPONSE TIME

Response time is a measure of system efficiency in handling various tasks, calculated as an average time between receiving user's query and producing relevant response. Low response time indicates effective task management to improve user experience.

$$\text{Avg Response time} = \frac{\text{Total Time Taken for All Tasks}}{\text{Number of Tasks}} \quad (2)$$

3) ROUGE SCORE

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) is a package of measures employed in the evaluation of automatic summarization and machine translation. It checks if a formed summary or translation is close enough to the reference set. ROUGE-N calculates the number of overlapping n-grams between system and reference summaries. ROUGE-L counts the length of longest common subsequence (LCS) between candidate and model summaries. It recognizes similarity of sentence-level structure as well as identifies automatically longest n-grams co-occurring in-sequence.

$$\text{ROUGE-N} = \frac{\sum_{S \in \{\text{Reference Summaries}\}} \sum_{\text{gram}_n \in S} \text{Match}(\text{gram}_n)}{\sum_{S \in \{\text{Reference Summaries}\}} \sum_{\text{gram}_n \in S} \text{Count}(\text{gram}_n)} \quad (3)$$

where:

gram_n: The n-grams in the reference summary.
$\text{Match}(\text{gram}_n)$: The maximum number of n-grams co-occurring in a candidate summary.

$$\text{ROUGE-L} = \frac{(1+\beta^2) R_l P_l}{R_l + \beta^2 P_l} \quad (4)$$

where:

R_l is the LCS-based recall
P_l is the LCS-based precision
β is the factor to balance precision and recall

4) BERT SCORE

In order to determine the similarity between two sentences, BERT Score makes use of pre-trained contextual embeddings from BERT models through a sum of cosine similarities of their tokens. This is done so as to capture semantic similarity beyond exact word matches thus making it more reliable in evaluating text generation tasks. BERT is used to evaluate the Q/A capabilities of the LLM module.

$$\text{BERT-F1} = 2 \cdot \frac{\text{BERT-P} \cdot \text{BERT-R}}{\text{BERT-P} + \text{BERT-R}} \quad (5)$$

$$\text{BERT-P} = \frac{1}{|x|} \sum_{x_i \in x} \max_{y_j \in y} x_i^T y_j \quad (6)$$

$$\text{BERT-R} = \frac{1}{|y|} \sum_{y_j \in y} \max_{x_i \in x} x_i^T y_j \quad (7)$$

In this formula:

x are the tokenized candidate
y are the reference sentences
x_i are the BERT embeddings of tokens
y_i are the BERT embeddings of tokens
$|x|$ is the length of the candidate sentence

5) USER SATISFACTION

Users' satisfaction was determined mainly by finding out whether the system delivered what the user expected in regard to their particular query. In the case of file search, users are satisfied based on the fact whether the file action has been completed or not. This could mean searching for a file, retrieving it or performing file operations by from the user. For summarization tasks it depends on how good the summary produced by the LLM is in relation to the one that was written by a human. A function is written for the LLM to self assess the summary w.r.t the summary written by human considering its relevance, completeness and clarity.

B. EXPERIMENTAL SETUP

To analyze the framework proposal, a group of 50 queries were chosen. These queries spanned four key categories: activities include file search, document management, code debugging, and a variety of general tasks. The design of every query sought to reflect the multiple ways users commonly interact with automation agents, ensuring a wide review of the system's performance. Tasks involving file searches tested the system's ability to identify files based on either their names or their content, just as someone would look for a single document in a messy workspace. Document management tasks included activities like summarizing reports or retrieving essential data, which resembles the act of reading a book for important excerpts. The system was put to the test by code debugging queries aimed at identifying errors or accessing code segments. For evaluating the summarization tasks, the inputs included human-written summaries compared to those generated by the LLM. The model was tested on a machine that has 8GB RAM and a Ryzen 5 4600H processor and the testing was done in the absence of a GPU. All the inputs and summaries used for evaluation are provided in the Github repo.

C. AGENT EVALUATION

RQ1: Does the agent classification system efficiently and accurately categorize tasks for execution?

Concerning this, A series of 50 experimental queries were used to assess the accuracy and response time of the agent classification system. Scenarios included task categorization for file searching, document handling, and general queries. The exact prompts could be found at the Github repo. All test cases were scored and assessed using metrics presented in Section IV-A to get final statistical results.

The agent classification system has shown high accuracy with 86% correct tasks categorization. On average, it takes 1.2 seconds per task as it classifies promptly.These findings indicate that out of these fifty queries that were tested, seven were places, where general tasks were misclassified as summarization task (SA).

A rate of misclassification of 14% (7 incorrectly classified queries out of 50) implies that the system will fail to allocate the correct category for the task, especially when user queries are more vague or more complicated. At this point however, all such misclassified queries are in the general tasks category while inappropriately taking up the document summarization module. The main concern in such instances is the likelihood of receiving incorrect output that is not pertinent to the

purpose intended, because a task was incorrectly assigned to an appropriate module.

There have been no significant cases where technical accuracy did not align with user satisfaction. For the classification agent, user satisfaction is straightforward; tags applied to each query are more often than not unambiguous. For instance, file related queries are tagged ''FS'', document or summarization related queries are tagged ''SA''. By categorizing tasks with this clarity, the risk of misclassification is reduced, and expectations of the user are always fulfilled.

Answer to RQ1: The agent classification systems successfully classify tasks with an accuracy rate of 86% and response time efficiency of 1.2 seconds; therefore, proving that the system is effective in providing reliable results.

D. RETRIEVER EVALUATION

RQ2: How effective is the retriever module in identifying and retrieving pertinent past interactions to aid present task responses?

The performance of the retriever module was evaluated by focusing on its ability to search for and fetch relevant old conversations. The evaluation of the module was conducted via comparing the retrieved conversations with the ground truth. The ground truth was created by designing controlled conversation scenarios in which the structure and content of previous interactions were known. For instance queries like ''give me invoice_shop.pdf'' were issued, and then made sure that there were indeed such files in the folder but not exact correspondences (for instance, other invoices). This served as a way of establishing whether the system was able to extract the appropriate historical conversation context from the system based on the meaning of the current query. The ground truth then comprised of planned interactions scenarios and various by retrieval outputs.

Good retrieval precision was achieved when there was high semantic reason which is one or more meanings found in the content as devised by instructions. The results confirmed that the retriever module was working within the anticipated parameters allowing relevant information to be drawn from the conversations that had been held before. Detailed conversation logs and ground truth has been provided in the Github repo.

Answer to RQ2: The retriever module performs well in recognizing and locating past interactions that are significant. This means it has the potential to build upon prior user engagements to improve current task responses. Without connecting to prior significant interactions the system is able to change by introducing fresh contexts or applying widespread knowledge to satisfy the users' needs.

E. SUMMARIZER EVALUATION

RQ3: Compared to expected benchmarks, how does overall system performance in terms of document summarization and how does it fare in terms of consistency, relevance and user satisfaction?

The assessment of how well the documents were handled was done based on the quality of summaries provided by the system and which was established using ROUGE as well as BERT scores. These metrics both evaluated the system's ability to produce good quality summaries that are coherent, well-structured and relevant to the contexts.

TABLE 2. ROUGE Score Comparison between GPT-4 and Proposed Model.

		ROUGE Scores		
Model	Summary	ROUGE-1	ROUGE-2	ROUGE-L
GPT-4	1	0.488	0.230	0.488
	2	0.511	0.163	0.511
Proposed	1	0.473	0.161	0.430
	2	0.515	0.218	0.474

The performance of the proposed model and GPT-4 along the two benchmarks for the two approaches is presented in Tables 2 and 3.

From the results outlined in Table 2, it is noted that the proposed model had positive ROUGE scores, and notably ROUGE-1 and ROUGE-L F-scores of 0.473 and 0.515 respectively over different summaries. This demonstrates that the proposed model is able to recognize unigrams and the gross structure of the text. The score for ROUGE-2 showed an acceptable sight of 0.161 and 0.218 which was clearly lower than all other too 'reasonable' hence the ability to recognize bi-grams which are critical for relations in texts at word pair level is still very much in place.

TABLE 3. BERT Score Comparison between GPT-4 and Proposed Model.

	BERT Score	
Model	Summary 1	Summary 2
GPT-4	0.9227	0.9189
Proposed	0.9243	0.9216

A high BERT(Table 3) scoring was recorded by the proposed model for both summaries presenting a validation for the semantic content of the generated summary. The data also recorded that the summaries generated by the proposed model were not only lexically similar to the reference texts but also proved to be semantically accurate with the reference text.

TABLE 4. Model performance comparison based on given criteria.

	Summary 1		Summary 2	
Criteria	GPT-4	Proposed	GPT-4	Proposed
Coherence	5.0	5.0	3.0	4.0
Consistency	5.0	5.0	5.0	5.0
Fluency	3.0	3.0	2.0	3.0
Relevance	5.0	4.0	4.0	4.0

Although GPT-4 performed marginally better in some of the metrics, the results of the proposed model reaffirm as it is

able to produce relevant and accurate summaries, even when resources are limited.

The summaries were also self assessed with respect to human written ones for coherence, consistency, fluency and relevance. The algorithm for measuring these metrics draws from the assessment strategy from OpenAI's cookbook. Both proposed model and GPT-4 were analyzed through this method for comparison on how well they perform.

The Proposed model and GPT-4 delivered comparable results in both coherence and consistency. The coherence rating for Summary 2 (4.0 vs. 3.0 for GPT-4) revealed that proposed model achieved a more favorable score greater than GPT-4. In Summary 1 both models generated equally fluid language scoring 3.0 each but GPT-4 faced issues in Summary 2 obtaining a much lower fluency score of 2.0 versus the proposed model's 3.0. The model scored 4.0 for relevance in Summary 2 whereas GPT-4 achieved a 5.0 in Summary 1. These results show that the proposed model is on par with GPT-4 in multiple significant categories.

To evaluate long form summarization, three chapters of Gulliver's Travels were summarized. The performance of the model was assessed using ROUGE and BERT scores, with results showing promising outcomes: A BERT score of 0.9410 along with ROUGE-1 at 0.6838 and ROUGE-2 at 0.29 along with ROUGE-L as 0.4286 were obtained.

Evaluations showed that BERT and ROUGE scores are very close to human evaluations in normal summarization tasks, there were no significant discrepancy. Yet in long form summarization, especially when compressing into multiple chapters, bigrams (or key phrase level detail) were sometimes left out. The problem of condensing complex information is a common one, and when summarizing, the summarizer is concerned with high level content at the expense of some of the detailed relationships between phrases. But that doesn't mean that the summary generated is inaccurate, it says that the generated summary won't have the specific phrases used in the original input. To address this, chapter-wise summarization was performed, and the results were then concatenated together. This would help the summarizer generalize better. Long form summarization was designed to give an overview of the text provided.

There have been no significant cases where technical accuracy did not align with user satisfaction in terms of summarizations. To ensure this the model's output is compared against human written summaries and then self assessed by the LLM. This ensures makes sure that summaries generated by the model are highly accurate, fluent and relevant to both technical metrics (e.g. ROUGE and BERT scores) and human expectations.Users can also express subjective preferences for their summaries in their input query in cases where they have a want for specific content to include or a limitation on the length of their summaries. When generating the summary, the system considers these preferences, to yield personalized outputs.

Answer to RQ3: With regard to the overall system performance in document summarization, it was above average even for the model-based language generation tools. Positive ROUGE scores were also obtained ROUGE-1: 0.473, ROUGE-L: 0.515 indicating the success of coherence and the overall structure. The high BERT scores also justified the coherency of the summary semantically. Although there was a slight edge gained by GPT-4 in some metrics, the model proved to be capable of and pertinent in summary generation especially when resources are limited. A qualitative analysis established the fact that the summaries were coherent, consistent, fluent in language, high relevant, and thus confirmed the effectiveness of the system.

V. DISCUSSION

By assessing the framework to implement local Large Language Models (LLMs) within a file system, promising results highlighted their potential for automating in-system tasks. Here, the focus is on synthesizing the fundamental observations, examining how the results have affected outcomes, and discussing their by extension importance in the scope of LLM-based systems.

Research goals were met by creating a privacy-oriented framework that permitted on-device handling of personal data using large language models. A structured design was created to separate various tasks into separate components like file search and document summarization. Analytical results demonstrated the system's effectiveness as the agent classification system reached 86% precision in one second and had a 14% error rate on general tasks being wrongly labeled as document summaries. The retriever module acted efficiently and consistently retrieved important information over time to boost user interactions for complex tasks. Strong summarization emerged from the system as evidenced by its high ROUGE-1 and ROUGE-2 scores of 0.6838 and 0.29 with scores of 0.4286 and 0.9410 for ROUGE-L and BERT.

The efficiency of the system relies on lengths of input and output tokens and the mechanism that allows recalling prior interactions at the upper limit imposed for the context window. The growth of tokenized queries resulting from a single input query leads to an extended time taken to perform the task and greater processing power to support new tasks.

To optimize system performance under heavy task loads, several functions have been implemented to enable the system to handle tasks efficiently. First is the token limit in summarization tasks. The input query is kept below model's context window (e.g. 5,000 tokens) to keep processing information as efficient as possible while maintaining accuracy. This prevents the model from ever performing poorly due to the lack of context, or because of performance bottlenecks from exceeding token limits. However, since the system is meant to process queries in the sequential order in a conversational way, where the user provides an input and gets a response before submitting another query, handling multiple queries at once is not required. Multithreading is used for faster internal processing in tasks such as file search, where scripts or commands generated by the model are

executed. It optimizes the system's response time for file operation which means that even complex query is executed fast without hampering flow of conversation. With respect to thresholds, it has been observed that performance can degrade if the summarizer reaches its context limit. In such scenario, response time may increase and output quality may be affected. But for these cases long form summarization has been implemented.

VI. CONCLUSION

A framework was developed for leveraging Local Large Language Models (LLMs), with the aim of promoting in-system security automation. The system employs Mistral 7B model to perform various tasks like file searching, document processing, code debugging and general tasks. Specialized modules were developed for each task type to ensure high precision and efficiency when addressing user queries.The File Searching Module effectively retrieves files, streamlining file access for users. The Document-Related Tasks Module demonstrated proficiency in summarization, question answering, and long-form summarization, providing comprehensive document analysis. The Code Debugging Module harnessed the inherent capabilities of LLMs towards assisting developers to identify and solve code problems. Lastly, the Generalized Tasks Module was enhanced by the Self-Knowledge Retrieval (SKR) which reduced hallucinations by making LLM self-assess its knowledge thus improving response reliability. The design and implementation of the system emphasize the promise of local LLMs in enhancing task automation and improving user interaction with computers.

VII. FUTURE WORKS

Furthermore, there are several areas that have potential for improvement and research. One possible enhancement is in the Classifying agent where complex queries can be classified to multiple categories and the system must be able to categorize the query into the specified tags. One way of reducing misclassifications is to implement an adaptive feedback loop based on user corrections. Extension of this work might consider integrating internet access at the same time upholding the privacy of user to answer queries that may require more up to date information to answer. This makes it more versatile and answer more user queries. Also building a user interface could enhance the usability of the system and make the interaction more intuitive.

In conclusion, this research sets a firm basis for exploiting local LLMs in task automation which have some potentials. Going forward, further developments and studies in this area can bring about changes to the interactions and use of computer systems.

REFERENCES

[1] Y. Wu, H. Iso, P. Pezeshkpour, N. Bhutani, and E. Hruschka, "Less is more for long document summary evaluation by LLMs," 2024, *arXiv:2309.07382*.
[2] M. Hassid, T. Remez, J. Gehring, R. Schwartz, and Y. Adi, "The larger the better? Improved LLM code-generation via budget reallocation," 2024, *arXiv:2404.00725*.
[3] R. Haldar and J. Hockenmaier, "Analyzing the performance of large language models on code summarization," 2024, *arXiv:2404.08018*.
[4] *Claude 3.5 Sonnet Model Card Addendum*, Anthropic, San Francisco, CA, USA, 2024.
[5] S. Shahriar, B. Lund, N. R. Mannuru, M. A. Arshad, K. Hayawi, R. V. K. Bevara, A. Mannuru, and L. Batool, "Putting GPT-4o to the sword: A comprehensive evaluation of language, vision, speech, and multimodal proficiency," *Appl. Sci.*, vol. 14, no. 17, p. 7782, 2024.
[6] X. Wu, R. Duan, and J. Ni, "Unveiling security, privacy, and ethical concerns of ChatGPT," *J. Inf. Intell.*, vol. 2, no. 2, pp. 102–115, 2023.
[7] Y. Li, J. Shi, and Z. Zhang, "An approach for rapid source code development based on ChatGPT and prompt engineering," *IEEE Access*, vol. 12, pp. 53074–53087, 2024.
[8] C. Wang, Y. Yang, C. Gao, Y. Peng, H. Zhang, and M. R. Lyu, "Prompt tuning in code intelligence: An experimental evaluation," *IEEE Trans. Softw. Eng.*, vol. 49, no. 11, pp. 4869–4885, Nov. 2023.
[9] L. Zhong and Z. Wang, "Can ChatGPT replace StackOverflow? A study on robustness and reliability of large language model code generation," 2024, *arXiv:2308.10335*.
[10] S. Wu, J. Xie, J. Chen, T. Zhu, K. Zhang, and Y. Xiao, "How easily do irrelevant inputs skew the responses of large language models?" 2024, *arXiv:2404.03302*.
[11] G. Lu, S. B. Larcher, and T. Tran, "Hybrid long document summarization using C2F-FAR and ChatGPT: A practical study," 2023, *arXiv:2306.01169*.
[12] Y. Chang, K. Lo, T. Goyal, and M. Iyyer, "BooookScore: A systematic exploration of book-length summarization in the era of LLMs," 2024, *arXiv:2310.00785*.
[13] S. Ryu, H. Do, Y. Kim, G. G. Lee, and J. Ok, "Key-element-informed sLLM tuning for document summarization," 2024, *arXiv:2406.04625*.
[14] Y. Hicke, A. Agarwal, Q. Ma, and P. Denny, "ChaTA: Towards an intelligent question-answer teaching assistant using open-source LLMs," 2023, *arXiv:2311.02775*.
[15] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig, "Active retrieval augmented generation," 2023, *arXiv:2305.06983*.
[16] Y. Wang, P. Li, M. Sun, and Y. Liu, "Self-knowledge guided retrieval augmentation for large language models," 2023, *arXiv:2310.05002*.
[17] J. Tan, Z. Dou, Y. Zhu, P. Guo, K. Fang, and J.-R. Wen, "Small models, big insights: Leveraging slim proxy models to decide when and what to retrieve for LLMs," 2024, *arXiv:2402.12052*.
[18] S. Lee, J. Choi, J. Lee, M. H. Wasi, H. Choi, S. Y. Ko, S. Oh, and I. Shin, "Explore, select, derive, and recall: Augmenting LLM with human-like memory for mobile task automation," 2024, *arXiv:2312.03003*.
[19] C. Packer et al., "MemGPT: Towards LLMs as operating systems," 2024, *arXiv:2310.08560*.
[20] W. Wang, L. Dong, H. Cheng, X. Liu, X. Yan, J. Gao, and F. Wei, "Augmenting language models with long-term memory," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 1–14.
[21] K. Pandya and M. Holia, "Automating customer service using LangChain: Building custom open-source GPT chatbot for organizations," 2023, *arXiv:2310.05421*.
[22] Z. Duan, "Application development exploration and practice based on LangChain+ChatGLM+Rasa," in *Proc. 2nd Int. Conf. Cloud Comput., Big Data Appl. Softw. Eng. (CBASE)*, Nov. 2023, pp. 282–285.
[23] O. Topsakal and T. C. Akinci, "Creating large language model applications utilizing langchain: A primer on developing LLM apps fast," in *Proc. Int. Conf. Appl. Eng. Natural Sci.*, 2023, pp. 1050–1056.
[24] Y. Han, C. Liu, and P. Wang, "A comprehensive survey on vector database: Storage and retrieval technique, challenge," 2023, *arXiv:2310.11703*.
[25] T. Händler, "Balancing autonomy and alignment: A multi-dimensional taxonomy for autonomous LLM-powered multi-agent architectures," 2023, *arXiv:2310.03659*.
[26] Y. Talebirad and A. Nadiri, "Multi-agent collaboration: Harnessing the power of intelligent LLM agents," 2023, *arXiv:2306.03314*.
[27] A. Yang et al., "Qwen2 technical report," 2024, *arXiv:2407.10671*.
[28] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, "Mistral 7B," 2023, *arXiv:2310.06825*.

[29] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C. C. Ferrer, A. Grattafiori, W. Xiong, A. Défossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve, "Code llama: Open foundation models for code," 2024, *arXiv:2308.12950*.

[30] T. Junaid, D. Sumathi, A. N. Sasikumar, S. Suthir, J. Manikandan, R. Khilar, P. G. Kuppusamy, and M. Janardhana Raju, "A comparative analysis of transformer based models for figurative language classification," *Comput. Electr. Eng.*, vol. 101, Jul. 2022, Art. no. 108051.

[31] P. N. Chowdary, M. S. Akshay, V. S. Aravind, M. S. Aashish, G. V. N. S. L. V. Vardhan, and G. J. Lal, "A few-shot approach to dysarthric speech intelligibility level classification using transformers," in *Proc. 14th Int. Conf. Comput. Commun. Netw. Technol. (ICCCNT)*, Jul. 2023, pp. 1–6.

[32] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 5776–5788.

[33] P. Elumalaivasan, S. Suthir, S. Ravikumar, V. Pandiyaraju, and T. Munirathinam, "CBIR: Retrieval of similar images using median vector algorithm," in *Proc. Int. Conf. Green Comput., Commun. Conservation Energy (ICGCE)*, Dec. 2013, pp. 1–5.

[34] A. Karthikeyan, S. Jothilakshmi, and S. Suthir, "Colorectal cancer detection based on convolutional neural networks (CNN) and ranking algorithm," *Meas., Sensors*, vol. 31, Feb. 2024, Art. no. 100976.

[35] B. M. Gurusamy, P. K. Rengarajan, and P. Srinivasan, "A hybrid approach for text summarization using semantic latent Dirichlet allocation and sentence concept mapping with transformer," *Int. J. Electr. Comput. Eng. (IJECE)*, vol. 13, no. 6, p. 6663, Dec. 2023.

[36] A. Karthikeyan, S. Jothilakshmi, and S. Suthir, "Colorectal cancer classification based on deep ensemble model with self-adaptive training model," *Int. J. Biomed. Eng. Technol.*, vol. 44, no. 4, pp. 367–396, 2024.



SUTHIR SRIRAM received the B.Tech. degree in information technology from Anna University, Chennai, the M.Tech. degree (Hons.) in information technology from Sathyabama University, and the Ph.D. degree in computer science and engineering from M. S. University. He is currently an Assistant Professor (Selection Grade) with the Department of Computer Science and Engineering, Amrita School of Computing, Amrita Vishwa Vidyapeetham, Chennai Campus. With 17 years of extensive teaching and research experience across esteemed academic institutions. He has published more than 50 papers in reputed international journals, flagship conferences, and book chapters. He also holds a granted patent and has delivered numerous guest lectures at various academic institutions. His research interests include machine learning, data science, blockchain, computer vision, and network intelligence.



C. H. KARTHIKEYA is currently pursuing the B.Tech. degree in computer science and engineering with Amrita Vishwa Vidyapeetham, Chennai. He's proficient in front-end development and interface design. He has a keen interest in human–computer interaction and his contributions aim to simplify complex systems and improve user experiences. His current research interests include computer vision, machine learning, and natural language processing.



K. P. KISHORE KUMAR is currently pursuing the B.Tech. degree in computer science and engineering with Amrita Vishwa Vidyapeetham, Chennai. He has gained valuable industry experience as an Intern at Quantumics AI and F13 Technologies. At F13 Technologies, he worked on strategizing and planning the migration of IRS systems to AWS, contributing to cloud infrastructure optimization. At Quantumics AI, he focused on research and development, particularly enhancing generative AI using the RAG framework for developing a data-driven chatbot. His projects include creating an interview chatbot designed for competency assessment and feedback using generative AI. His research interests include artificial intelligence and cloud computing.



NIVETHITHA VIJAYARAJ received the B.E. degree in electronics and communication engineering from Anna University, Coimbatore, and the M.E. degree in communication systems from Anna University, Chennai. She is currently pursuing the Ph.D. degree with Vellore Institute of Technology (VIT), Chennai Campus. She is an Assistant Professor with the Department of Computer Science and Engineering, Amrita School of Computing, Amrita Vishwa Vidyapeetham, Chennai. With over ten years of teaching and research experience in reputed academic institutions and three years of industry experience. She holds a granted patent. She has published over ten papers in esteemed international journals and flagship conferences. Her research interests include optical communications, quantum computing, machine learning, and photonics.



THANGAVEL MURUGAN (Senior Member, IEEE) received the bachelor's (B.E.) degree (Hons.) in computer science and engineering from the M. A. M. College of Engineering, Trichy, under Anna University, Chennai, the M.E. degree (Hons.) in computer science and engineering from the J. J. College of Engineering and Technology, Trichy, under Anna University, and the Ph.D. degree from Madras Institute of Technology (MIT) Campus, Anna University. He is currently an Assistant Professor with the Department of Information Systems and Security, College of Information Technology, United Arab Emirates University. He holds 11 years of teaching and research experience from various academic institutions. He has published more than ten articles in international journals, more than 15 book chapters in international publishers, more than 25 in the proceedings of international conferences, and three in the proceedings of national conferences/seminars. His research interests include information security, high-performance computing, ethical hacking, cyber forensics, blockchain, cybersecurity intelligence, and educational technology. He received the Gold Medal for the B.E. and M.E. degrees.